



INGENIERÍA DE SOFTWARE
IASI0729 - DESARROLLO DE APLICACIONES OPEN SOURCE
EXAMEN FINAL
202610

Sección: 11990

Profesores: Bautista Ubillús, Efraín Ricardo
Flores Moroco, Juan Antonio
Mori Paiva, Hugo Allan
Robles Fernández, Iván
Velásquez Núñez, Ángel Augusto

Duración: 170 minutos

Indicaciones:

1. El examen consta de 1 pregunta, y tendrá **170 minutos** para resolverlas.
 2. La pregunta es de tipo Proyecto de Software y la entrega de su respuesta es a través de envío de archivo empaquetado **.zip** (único formato válido) con nombre eb<nrc>u<código-estudiante>.zip (por ejemplo, eb11990u201621873.zip), conteniendo el proyecto de software, en la Actividad para el Examen final.
 3. Debe crear su proyecto de solución y evidenciar la autoría.
 4. Puede utilizar como referencia los materiales publicados en el aula virtual, los sitios web de documentación de frameworks, lenguaje de programación utilizados, así como ejemplos de clase (solo como referencia, no como fuente de duplicado o copia).
-

Enunciado:

Caso APU Winery.

APU Winery (<https://www.apuwinery.com/>), a través de su plataforma inteligente de viticultura andina Terroir Shield, busca automatizar, monitorear y garantizar la trazabilidad de sus micro-lotes de vino de alta montaña y viticultura heroica cultivados en Curahuasi (Apurímac). Utilizando un ecosistema de microservicios distribuidos, la bodega busca controlar rigurosamente las variables de fermentación y el ciclo de asignación de botellas exclusivas para el sector gastronómico de alta gama, alineándose con las normativas locales de producción agrícola y sostenibilidad.

Pregunta 1 (20 p.).

Usted se integra al **backend software developer team** para crear un RESTful API de alta eficiencia que gestione las operaciones de monitoreo físico de fermentación y la asignación prioritaria de botellas de vino a restaurantes de alta gastronomía.

El ecosistema requiere que el contexto inteligente de viticultura maneje la información de:

WineBatches: Con atributos *id*, *batchCode*, *bottleEstimate*, y un *fermentationMetrics* Value Object (de tipo **Metrics** que contiene los valores químicos *phLevel* y *sugarDensity*). Adicionalmente, incluye el atributo *lastOrderId* (nullable) para el control de concurrencia y flujo de pedidos.

WineOrders: Con atributos *id*, *wineBatchId*, *sommelierNotes* (notas de cata y validación de reserva), *orderPriority* (de tipo *OrderPriority*), y *orderedAt*.

Como parte de las reglas de negocio APU Winery especifica que:

- El aggregate **WineBatch** contiene: **id** (Long, PK, obligatorio, autogenerado), **batchCode** (String, único, formato estricto: "APU-BATCH-XXXXX"), **bottleEstimate** (Integer, obligatorio, mínimo 50, máximo 1500), **fermentationMetrics** (*Metrics*, obligatorio), y **lastOrderId** (Long, nullable).
- El aggregate **WineOrder** contiene: **id** (Long, PK, obligatorio, autogenerado), **wineBatchId** (Long, obligatorio), **sommelierNotes** (String, obligatorio, debe comenzar estrictamente con la sentencia "RESERVA EXCLUSIVA:" seguida de un código de validación de 8 caracteres alfanuméricos), **orderPriority** (*OrderPriority*, obligatorio), y **orderedAt** (*LocalDateTime*, obligatorio, no mayor a la fecha y hora actual).
- **OrderPriority** es un enumeration con los valores: LOW, MEDIUM, HIGH, CRITICAL.
- El sistema debe gestionar la viabilidad comercial y de embotellado de los lotes de forma dinámica. El estado **ripeningStatus** no debe ser persistido en la base de datos, sino calculado en tiempo de ejecución como un invariante complejo del dominio: el lote se considerará en estado **READY** (Listo para embotellar) si el valor de **phLevel** es estrictamente menor o igual a 3.65 y el método de negocio *isReadyForBottling()* retorna verdadero. En caso de superar dicho umbral de acidez o estar bloqueado temporalmente por pedidos críticos en proceso, el estado devuelto debe ser **FERMENTING**.
- Para el registro de asignaciones o pedidos de vino (**WineOrders**), el sistema debe validar la existencia y maduración del lote asociado mediante una capa de Anti-Corruption Layer (ACL) que comunique el bounded context de **sales** con el de **winery**. Si el **ripeningStatus** calculado para dicho lote es **FERMENTING**, el registro del pedido debe ser rechazado inmediatamente lanzando una excepción de dominio traducida localmente. Al registrar un **WineOrder** válido, se debe emitir un evento de dominio distribuido llamado **OrderPlacedEvent** que contenga el **wineBatchId**, **orderId**, **orderPriority** y **orderedAt**.
- El *Winery Bounded Context* debe implementar un Event Handler para el evento **OrderPlacedEvent** que garantice la integridad y consistencia mediante las siguientes condiciones: El proceso de consumo debe ser estrictamente idempotente, verificando si el **orderId** ya ha sido registrado; debe ejecutarse dentro de una transacción única aislada para asegurar la atomicidad de la actualización del estado físico del lote; Debe aplicar *Optimistic Locking* en la entidad **WineBatch** utilizando un atributo de versión para prevenir inconsistencias ante accesos concurrentes masivos; y finalmente, como

mecanismo de control operativo, el handler debe actualizar el atributo **lastOrderId** del **WineBatch** aggregate. Mientras este atributo contenga un valor activo y la prioridad del evento sea **CRITICAL**, el método de negocio del dominio **isReadyForBottling()** del aggregate debe retornar **false**, bloqueando temporalmente el lote y simulando una saturación de preparación por despacho prioritario.

- La plataforma debe realizar un *Data Seeding* automático al iniciar la aplicación mediante el evento *ApplicationReadyEvent* de Spring Boot, poblando la información base de *WineBatches* con los siguientes datos iniciales:

id	batchCode	bottleEstimate	phLevel	sugarDensity
1	APU-BATCH-00011	350	3.50	4.25
2	APU-BATCH-00032	120	3.60	4.80
3	APU-BATCH-00064	800	3.82	5.10
4	APU-BATCH-00128	450	3.45	3.15

- Requiere que el valor de **orderedAt** no sea mayor que la fecha y hora actual del servidor.
- Al momento del request para la creación de un pedido, el sistema debe soportar que se proporcione el valor de **orderedAt** como un String estructurado bajo el patrón estricto "yyyy-MM-dd HH:mm:ss".
- Todos los responses del API deben soportar localization (i18n).
- Ambos Aggregates requieren contar con atributos de auditoría para registrar **createdAt** y **updatedAt**, poblados de forma automática por el framework ORM del sistema.

Durante la etapa de desarrollo, le asignan trabajar en específico sobre dos Endpoints:

/api/v1/wine-orders
/api/v1/wine-batches

Incluya como parte del desarrollo la implementación de todas las reglas de negocio.

Wine Orders Queue Endpoint (/api/v1/wine-orders)

Debe exponer solo una operación de escritura en el RESTful API: agregar (POST) una orden de vino. El valor del identificador único (id) no se solicita en el JSON de entrada, siendo autogenerado por la estrategia de persistencia relacional subyacente. Al agregar exitosamente el registro, se debe retornar un HTTP Status 201 (Created). El cuerpo de la respuesta (response JSON) debe incluir un objeto con el id generado y sus demás atributos mapeados en un formato plano, excluyendo explícitamente los atributos de auditoría interna (**createdAt**, **updatedAt**) y el estado calculado **ripeningStatus**.

Wine Batches Endpoint (/api/v1/wine-batches)

Debe exponer solo una operación de lectura en el RESTful API: recuperar (GET) el listado de lotes de vino actualmente registrados en el sistema de monitoreo. Debe retornar la colección completa de objetos almacenados. No incluya en las respuestas los atributos de auditoría.

Nota:

En esta evaluación, está permitido Code Completion suggestions por parte de IntelliJ IDEA, pero no está permitido el uso de herramientas o AI Agents para Code Generation. El uso de herramientas o AI Agents para code generation puede invalidar la evaluación.

Technical constraints

- Elabore la solución con Java 26, Spring Boot Framework 4.1 como development framework y Spring Data JPA como ORM.
- Cree su proyecto de software con el nombre **eb<NRC>u<código-estudiante>** (por ejemplo, **eb11990u201621873**).
- La información debe ser persistente en una base de datos relacional (PostgreSQL), en un esquema **apu_winery_db**.
- Los packages de su solución deben tener como nombre raíz **com.apuwinery.platform.u<código-estudiante>** (por ejemplo, **com.apuwinery.platform.u201621873**).
- Considere que el concepto **WineBatch** pertenece al bounded context **winery**, el concepto **WineOrder** pertenece al bounded context **sales** y el concepto **Metrics** pertenece al bounded context shared.

6. Considere el bounded context **shared** para elementos base comunes/reutilizables que puedan ser aprovechados o extendidos por elementos en otros bounded contexts. El contenido del bounded context shared debe seguir las especificaciones del bounded context *shared* del proyecto de ejemplo *learning center platform* revisado en clase.
7. Aplique buenas prácticas de Arquitectura de Software, enfoque de **Domain-Driven Design**, separación en bounded contexts, **layered architecture** (domain, application, interfaces, infrastructure), patrones de strategic y tactical Domain-Driven Design, patrón CQRS, patrón Anti-Corruption Layer (ACL), principios y patrones de diseño de software orientado a objetos, convenciones de nomenclatura en **inglés**, así como buenas prácticas de nomenclatura en Java (entre ellas Upper-Camel-Case para Clases, Lower-Camel-Case para atributos y métodos) y buenas prácticas para nomenclatura de objetos de Base de Datos (entre ellas snake case, tablas en plural, sin mnemónicos).
8. Aplique separación de *Domain Concepts* como *Aggregates* o *Domain Repositories* de *Persistence Concepts* como *Persistence Entities* (con el decorador @Entity) o *Persistence Repositories* y *Repository adapters*.
9. Aplique reglas de object-relational mapping, implementando en shared un physical naming strategy que establezca de forma automática las convenciones de snake case para objetos de base de datos, así como plural para nombres de tablas.
10. Utilice el patrón Assembler para el Object Mapping en la sección *transform* en interfaces layer.
11. Utilice minúsculas para los nombres de URL y términos compuestos separados por guión medio (-) para todos los endpoints.
12. Utilice la biblioteca Lombok para el manejo de métodos constructores y de acceso en las clases de Java.
13. Utilice records en vez de clases para almacenamiento de valores inmutables.
14. Para **wine-batch** e **wine-order**, incluya atributos de auditoría createdAt y updatedAt con valores poblados de forma automática por Spring Boot al momento de la creación.
15. Documente su código con **JavaDoc** (ver referencias), colocando en inglés información de propósito para principales objetos de programación, así como propósito, parámetros y valor retornado en clases y métodos relevantes. Incluya como parte de la documentación sus nombres y apellidos como valor para @author.
16. Incluya documentación de Endpoints con **OpenAPI**, proporcionando información a nivel general, de las operaciones, los requests y los posibles responses.
17. Por defecto, todos los textos de mensajes deben ser en inglés.
18. El API debe soportar los mensajes de error o validación en el response en los language Inglés (EN, EN-US) o Español (ES, ES-PE), según indique el parámetro **Accept-Language** en el header del request.
19. Considere la gestión de excepciones en la aplicación, proporcionando Responses adecuados.
20. Incluya en el archivo README.md, la información en inglés de la aplicación, descripción y su información como author.
21. Empaquete su solución como un archivo **.zip**. (**único formato válido**) con el nombre **eb<NRC>u<código-estudiante>.zip** (por ejemplo, *eb11990u201621873.zip*).
22. Suba su archivo de solución en la Actividad indicada para el Examen final.

NO forma parte del alcance del proyecto:

1. Soporte de CORS.
2. Security.
3. Testing.

Rúbrica de calificación

Criterio de Calificación	Sobresaliente (S)	Esperado (E)	Necesita Mejorar (M)	Insuficiente (I)	Calificación
C01. Building y ejecución	Al abrir el proyecto y ordenar la ejecución, ésta se inicia sin problemas. El API es accesible en la ruta indicada.	La aplicación no llega a iniciar y ejecutarse, sin embargo el proceso de building llega a concluir.	Al cargar el proyecto el proceso de building presenta errores y no llega a concluir.	No elabora solución.	
	2.0 puntos	1.0 punto	0.5	0 puntos	
C02. First Endpoint	El RESTful API expone el endpoint especificado en el enunciado. Se evidencia la funcionalidad de las operaciones solicitadas, proporcionando en cada caso los valores esperados y respondiendo adecuadamente ante las excepciones. Se evidencia la persistencia de los objetos solicitados, cumpliendo la estructura según enunciado, en una tabla nombrada según especificaciones, en base de datos relacional según enunciado en el esquema indicado. Se incluye documentación del Endpoint con OpenAPI.	El RESTful API expone el endpoint especificado en el enunciado. Se evidencia parcialmente la funcionalidad de las operaciones solicitadas, proporcionando en algunos casos los valores esperados y respondiendo de forma parcialmente adecuada ante las excepciones, o se evidencia parcialmente la persistencia en base de datos relacional, o se evidencia parcialmente la persistencia de los objetos solicitados con estructura según enunciado, en una tabla <i>nombrada según especificaciones</i> en base de datos relacional según enunciado en el esquema <i>indicado</i> .	La aplicación implementa y expone el endpoint especificado en el enunciado, pero no cumple con la ruta especificada o no se puede registrar elementos.	La aplicación no implementa o expone el endpoint solicitado.	
	4.0 puntos	3.0 puntos	1.5 puntos	0 puntos	
C03. Second Items Endpoint	El RESTful API expone el endpoint especificado en el enunciado. Se evidencia la funcionalidad de las operaciones solicitadas, proporcionando en cada caso los valores esperados y respondiendo adecuadamente ante las excepciones. Se evidencia la persistencia de los objetos solicitados, cumpliendo la estructura según enunciado, en una tabla nombrada según especificaciones, en base de datos relacional según enunciado en el esquema indicado. Se incluye documentación del Endpoint con OpenAPI.	El RESTful API expone el endpoint especificado en el enunciado. Se evidencia parcialmente la funcionalidad de las operaciones solicitadas, proporcionando en algunos casos los valores esperados y respondiendo de forma parcialmente adecuada ante las excepciones, o se evidencia parcialmente la persistencia en base de datos relacional, o se evidencia parcialmente la persistencia de los objetos solicitados con estructura según enunciado, en una tabla <i>nombrada según especificaciones</i> en base de datos relacional según enunciado en el esquema <i>indicado</i> .	La aplicación implementa y expone el endpoint especificado en el enunciado, pero no cumple con la ruta especificada o no se puede registrar elementos.	La aplicación no implementa o expone el endpoint solicitado.	
	4.0 puntos	3.0 puntos	1.5 puntos	0 puntos	
C04. Business Rules	El desarrollo incluye la implementación de reglas de negocio, cubriendo de forma completa las condiciones y escenarios establecidos, siendo éstas ejecutables, con adecuado manejo de excepciones, implementando éstas en las capas más adecuadas, aplicando convenciones y buenas prácticas.	El desarrollo incluye la implementación de la mayoría de reglas de negocio, cubriendo de forma parcial las condiciones y escenarios establecidos, siendo éstas ejecutables, implementándolas en las capas adecuadas en la mayoría de casos, ó aplicando parcialmente convenciones y buenas prácticas.	El desarrollo incluye la implementación de algunas de las reglas de negocio, incumpliendo la mayoría de condiciones y escenarios establecidos, siendo éstas ejecutables, implementándolas en las capas adecuadas en muy pocos casos, ó con poca evidencia de aplicar convenciones y buenas prácticas.	No implementa reglas de negocio, o no cubre escenarios más allá de operaciones CRUD básicas, o éstas no son ejecutables.	
	4.0 puntos	3.0 puntos	1.5 puntos	0 puntos	
C05. Code Organization	El desarrollador organiza el código y los elementos de backend de la solución, aplicando buenas prácticas de Java, Spring Boot Framework y Domain-Driven Design, agrupando los elementos de la solución según convenciones, manteniendo organización de paquetes y carpetas recomendadas por el fabricante y buenas prácticas de la industria de software.	El desarrollador aplica en la mayoría de casos para el backend convenciones, recomendaciones y buenas prácticas de Java, Spring Boot Framework y Domain-Driven Design.	El desarrollador aplica en algunos casos para el backend convenciones, recomendaciones y buenas prácticas de Java, Spring Boot Framework y Domain-Driven Design.	No se evidencia un criterio de organización para los elementos de la solución.	
	2.0 punto	1.0 puntos	0.5	0 puntos	
C06. Code Quality	Utiliza para el backend el lenguaje de programación Java. La codificación tiene un estilo claro, indentando los bloques de código según los estándares de programación correspondientes al lenguaje, aplicando una lógica consistente en los métodos, condicionales sin escenarios no contemplados, uso adecuado de reutilización de código para evitar redundancia. Aplica patrones de arquitectura y patrones de diseño. Distribuye el código en los niveles correspondientes, asignando lógica de persistencia, lógica de negocio, lógica de control, lógica de mapping y transferencia a las interfaces y clases que corresponden. Cumple de forma completa con los technical constraints.	Utiliza para el backend el lenguaje de programación Java. La codificación es funcional, aplica en la mayoría de casos los estándares de indentación de bloques de código, ó existen algunas ineficiencias en la codificación: redundancia ó inconsistencias en la lógica de programación. Aplica parcialmente patrones de arquitectura y patrones de diseño, o existe en algunas partes una distribución de la lógica en los niveles incorrectos. Cumple con la mayoría de technical constraints.	Utiliza para el backend el lenguaje de programación Java. La codificación es funcional, pero solo aplica algunos de los estándares de indentación de bloques de código, ó existen muchas ineficiencias en la codificación: redundancia ó inconsistencias en la lógica de programación. Aplica algunos patrones de arquitectura y patrones de diseño, o existe en muchos casos una distribución de la lógica en los niveles incorrectos. Cumple con solo algunos de los technical constraints.	No utiliza el lenguaje de programación Java para el backend, ó la codificación es funcional pero no se evidencia aplicación de estándares ó criterios de eficiencia en la codificación, con ausencia de comentarios, ó no aplica patrones de arquitectura ni patrones de diseño, o la codificación no es funcional.	
	3.0 puntos	2.0 punto	1.0	0 puntos	
C07. Naming Standards	El desarrollador aplica en todos los nombres de objetos de programación y base de datos como paquetes, componentes, interfaces, clases, objetos, variables, constantes, métodos, tablas, columnas la nomenclatura en inglés y la nomenclatura estándar para identificadores de clases, objetos, miembros de programación, tablas, columnas, así como los recursos.	El desarrollador aplica en la mayoría de casos la nomenclatura en inglés y la nomenclatura estándar para identificadores de clases, objetos, miembros de programación, así como los recursos.	El desarrollador aplica en muy pocos casos la nomenclatura en inglés y la nomenclatura estándar para identificadores de clases, objetos, miembros de programación, así como los recursos.	El desarrollador no aplica nomenclatura en inglés para los objetos de programación ó recursos.	
	1.0 puntos	0.5 punto	0.25 puntos	0 puntos	
Total	20 puntos	13.5 puntos	6.5 puntos	0 puntos	

Lima, 13 de julio del 2026

Anexos

Anexos A. Referencias

Comprimir y descomprimir archivos:

<https://support.microsoft.com/es-es/windows/comprimir-y-descomprimir-archivos-8d28fa72-f2f9-712f-67df-f80cf89fd4e5>

REST API Tutorial:

<https://restfulapi.net/>

Project Lombok:

<https://projectlombok.org/>

springdoc-openapi:

<https://springdoc.org/>

Spring Data JPA - Reference Documentation:

<https://docs.spring.io/spring-data/jpa/docs/current/reference/html/#reference>

Class UUID:

<https://docs.oracle.com/en/java/javase/25/docs/api/java.base/java/util/UUID.html>

Online UUID Generator:

<https://www.uuidgenerator.net/version4>

How to Write Doc Comments for the Javadoc Tool:

<https://www.oracle.com/technical-resources/articles/java/javadoc-tool.html>

Formats for date and dateTime in JSON payloads

<https://www.geeksforgeeks.org/formats-for-date-and-datetime-in-json-payloads/>

Get Date and Time From a Datetime String in Java

<https://www.baeldung.com/java-date-time-from-string>

Spring Data JPA Auditing:

<https://docs.spring.io/spring-data/jpa/reference/auditing.html#auditing.annotations>

How to validate MAC address using Regular Expression

<https://www.geeksforgeeks.org/dsa/how-to-validate-mac-address-using-regular-expression/>